

STACKBURGER

Aplicación Móvil · Delivery & Takeaway

DOCUMENTACIÓN TÉCNICA

Entrega 1 — MVP Básico

Este documento detalla las decisiones tecnológicas tomadas para la construcción del MVP de StackBurger: framework elegido, librerías y versiones, estrategia de navegación y diagrama de flujo entre pantallas.

Proyecto StackBurger App	Framework React Native 0.85.3	Plataforma Android (API 24+)	Node.js ≥ 22.11.0
------------------------------------	---	--	-----------------------------

- ☒ Decisión de framework justificada
- ☒ Librerías y versiones documentadas
- ☒ Gestión de estado definida
- ☒ Diagrama de navegación completo

1. Decisión Tecnológica

1.1 Framework elegido: React Native

El equipo eligió React Native como framework de desarrollo por las siguientes razones:

¿Por qué React Native y no Android Nativo (Kotlin)?

Ventajas determinantes para este proyecto:

- Código compartido entre plataformas: una sola base de código funciona en Android e iOS, lo que facilita escalar la app en E3 sin duplicar esfuerzo.
- Hot reload: permite ver cambios en la UI al instante durante el desarrollo sin recompilar toda la app, acelerando significativamente el ciclo de iteración.
- Ecosistema JavaScript/TypeScript: el equipo tiene experiencia previa en JS, lo que reduce la curva de aprendizaje respecto a Kotlin.
- React Navigation: librería madura y bien documentada para la gestión de flujos de pantallas, con soporte activo y amplia comunidad.
- Escalabilidad hacia E2/E3: la arquitectura de componentes de React facilita la incorporación de autenticación (Firebase Auth), gestión de estado avanzada y módulos adicionales sin refactoring mayor.

1.2 Versión mínima de Android soportada

La app soporta Android desde API 24 (Android 7.0 Nougat, lanzado en 2016) en adelante. Esta decisión se basa en:

- Cobertura: API 24+ representa más del 97% de los dispositivos Android activos según los datos de distribución de Google Play (2024).
- Compatibilidad: React Native 0.85.3 requiere como mínimo API 24 para garantizar el correcto funcionamiento de sus APIs nativas.
- Librerías: react-native-screens y react-native-gesture-handler tienen soporte garantizado desde API 24.

1.3 Lenguaje: TypeScript

El proyecto utiliza TypeScript (v5.8.3) sobre JavaScript por las siguientes razones:

- Tipado estático que previene errores en tiempo de compilación, especialmente útil en estructuras de datos como los productos del carrito.
- Mejor autocompletado e intellisense en el editor, lo que acelera el desarrollo.
- El template oficial de React Native CLI ya incluye soporte TypeScript de forma nativa.

2. Librerías y Dependencias

A continuación se detallan todas las dependencias del proyecto extraídas del package.json real:

Paquete	Versión	Tipo	Rol en el proyecto
<code>react-native</code>	0.85.3	Producción	Framework principal de la aplicación móvil.
<code>react</code>	19.2.3	Producción	Librería base de UI sobre la que corre React Native.
<code>@react-navigation/native</code>	^7.2.4	Producción	Núcleo del sistema de navegación entre pantallas.
<code>@react-navigation/stack</code>	^7.9.2	Producción	Stack Navigator: navegación tipo pila entre pantallas.
<code>react-native-screens</code>	^4.25.2	Producción	Optimiza el rendimiento de las pantallas usando componentes nativos.
<code>react-native-gesture-handler</code>	^2.31.2	Producción	Manejo nativo de gestos táctiles requerido por React Navigation.
<code>react-native-safe-area-context</code>	^5.8.0	Producción	Manejo de áreas seguras en pantallas con notch o bordes redondeados.
<code>@react-native-masked-view/masked-view</code>	^0.3.2	Producción	Dependencia requerida por el Stack Navigator para animaciones de transición.
<code>typescript</code>	^5.8.3	Dev	Tipado estático sobre JavaScript.
<code>@react-native-community/cli</code>	20.1.0	Dev	CLI oficial para correr, buildear y gestionar el proyecto.
<code>eslint + prettier</code>	^8.19.0 / 2.8.8	Dev	Linting y formateo de código para mantener estilo consistente.
<code>jest</code>	^29.6.3	Dev	Framework de testing unitario.

Nota sobre dependencias

No se incluyen librerías de UI externa (ej: React Native Paper, NativeBase) en E1. La UI se construye con componentes nativos de React Native para mantener el bundle liviano y el control total del diseño. Se evaluará incorporar una librería de componentes en E2 si la complejidad de la interfaz lo justifica.

3. Gestión de Estado

3.1 Estrategia elegida: Context API + useReducer

Para E1 se utilizará Context API nativa de React junto con el hook useReducer para gestionar el estado global del carrito. No se incorpora ninguna librería externa de manejo de estado (Redux, Zustand, Jotai).

¿Por qué Context API y no Redux?

- Simplicidad para E1: el estado global en E1 se reduce al carrito de compras. La complejidad de Redux (actions, reducers separados, store, middleware) no está justificada para este alcance.
- Sin dependencia adicional: Context API es parte del core de React, por lo que no agrega peso al bundle ni requiere configuración extra.

- Escalabilidad suficiente: si en E2 o E3 el estado crece en complejidad (sesión de usuario, múltiples módulos), se puede migrar a Zustand con cambios mínimos dado que la arquitectura de reducers es compatible.
- Curva de aprendizaje: el equipo ya conoce React hooks, lo que permite implementar esta solución rápidamente.

3.2 Estructura del estado del carrito

El estado global administrado por el contexto en E1 tiene la siguiente forma:

```
// Estructura del estado del CarritoContext
{
  items: [
    {
      id: string,           // ID único del ítem en el carrito
      productId: string,    // ID del producto en el catálogo
      nombre: string,
      precio: number,
      cantidad: number,
      extras: {
        medallones: number, // 1 a 5
        cheddar: number,   // 0 a 5
        panceta: number,    // 0 a 5
      },
      aclaracion: string,   // máx. 150 caracteres
    },
  ],
  total: number            // calculado automáticamente
}
```

3.3 Acciones del reducer

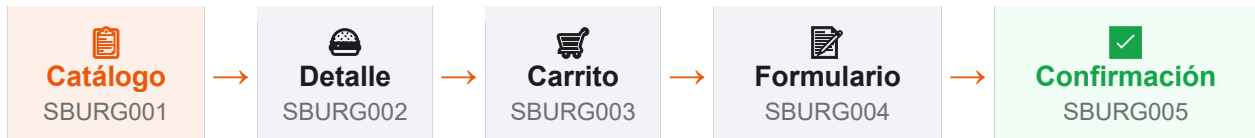
Las acciones disponibles sobre el carrito son:

Acción	Descripción
AGREGAR_ITEM	Añade un nuevo producto personalizado al carrito y recalcula el total.
MODIFICAR_CANTIDAD	Actualiza la cantidad de un ítem existente. Si llega a 0, lo elimina.
ELIMINAR_ITEM	Elimina un ítem del carrito por su ID y recalcula el total.
VACIAR_CARRITO	Limpia todos los ítems del carrito. Se ejecuta tras una confirmación exitosa.

4. Diagrama de Navegación

4.1 Stack Navigator — flujo principal E1

La aplicación usa un Stack Navigator de React Navigation. El usuario avanza linealmente por las pantallas del flujo de pedido y puede retroceder usando el botón nativo de Android o la flecha de la app:



4.2 Descripción de cada pantalla

ID	Pantalla	Componente	Descripción y estados
SBURG001	Catálogo	CatalogoScreen	Lista de productos. Estados: Normal (listado), Carga (skeleton), Error (botón reintentar), Vacío.
SBURG002	Detalle	DetalleScreen	Detalle del producto seleccionado con opciones de personalización (extras, cantidad, aclaración). 1 estado principal.
SBURG003	Carrito	CarritoScreen	Lista de ítems añadidos con subtotales y total. Estados: Normal (con ítems), Error (si falla algo), Vacío (sin ítems).
SBURG004	Formulario	FormularioScreen	Formulario de datos del cliente (Nombre, Apellido, Teléfono, Dirección). Validación en tiempo real. 1 estado principal.
SBURG005	Confirmación	ConfirmacionScreen	Pantalla de éxito. Muestra resumen del pedido y limpia el carrito. 1 estado principal.

4.3 Reglas de navegación

- La navegación es siempre hacia adelante en el flujo principal (Catálogo → Detalle → Carrito → Formulario → Confirmación).
- El botón físico de Android y la flecha del header permiten retroceder en cualquier punto, excepto desde la pantalla de Confirmación (el carrito ya fue limpiado).
- El ícono del carrito en el header de Catálogo y Detalle permite saltar directamente a CarritoScreen sin perder el estado.
- Desde Confirmación, el botón "Volver al menú" hace un reset completo del stack y navega a CatalogoScreen.

Deuda técnica identificada para E2

Los siguientes puntos se resolverán en la Entrega 2:

- Incorporar Tab Navigator para la navegación entre Catálogo, Carrito e Historial una vez que exista autenticación.
- Agregar Drawer Navigator para el acceso al perfil de usuario y configuración de cuenta.
- Evaluar migración de Context API a Zustand si la complejidad del estado global lo requiere.
- Documentar endpoints de la API REST una vez definido el backend (Firebase o servidor propio).